Algorithm–Hardware Co-Design of Digital Compute-in-Memory Architecture Supporting Flexible and Temporal N:M Sparsity

Akshat Ramachandran, *Student Member, IEEE*, Souvik Kundu, Arnab Raha, *Senior Member, IEEE*, Shamik Kundu, Deepak A. Mathaikutty, *Member, IEEE*, and Tushar Krishna, *Senior Member, IEEE*

***Abstract*—Structured pruning with fixed N:M sparsity ratios in large language models (LLMs) significantly constrains model expressivity, often leading to suboptimal accuracy. While supporting multiple N:M configurations can enhance representational flexibility, such a capability typically introduces substantial hardware complexity and overhead. To overcome these limitations, we first present FLOW, a flexible, layer-wise, outlier-density-aware N:M sparsity selection framework. FLOW adaptively determines the optimal N and M values per layer within a specified range by jointly considering the magnitude and distribution of outliers, thereby improving sparsity allocation and model fidelity. Extending this idea, we introduce FLOW++, which generalizes flexible N:M sparsity to the temporal domain for reasoning LLMs. FLOW++ enables temporal pruning by dynamically adapting sparsity patterns across reasoning thought types. To support efficient deployment of models with such dynamically varying sparsity patterns, we propose FlexCiM, a flexible, low-overhead, digital compute-in-memory (DCiM) architecture. FlexCiM partitions the DCiM macro into smaller submacros, which are adaptively aggregated and disaggregated through distribution and merging mechanisms for different values of N and M. We conduct experiments across different LLM families and state-space models and conclusively demonstrate that the proposed algorithm-hardware co-design framework achieves up to 36% higher accuracy, 1.75× faster inference, and 1.5× lower energy consumption compared to existing alternatives, establishing an effective balance between flexibility and hardware efficiency in sparse LLM inference. Code is available at: https://github.com/FLOW-open-project/FLOW**

***Index Terms*—Compute-in-memory, dynamic N:M sparsity, efficient inference, sparse large language model (LLM) accelerator, sparsity.**

I. Introduction

TO COMPRESS the overwhelming memory footprint of modern large language models (LLMs) and make them deployable on devices with limited compute and memory resources, post-training pruning has emerged as one of the central model–compression strategies [6], [13], [24], [35]. Pruning reduces the memory footprint of the pretrained LLMs by removing ineffectual model parameters, at the granularity of individual weights (unstructured) or blocks of weights (structured), and storing sparse tensors in a compressed format [CSR/compressed sparse column (CSC)] [10]. Beyond memory savings, pruning can also reduce compute and energy costs by skipping operations involving zeroed weights or activations.

However, classical pruning techniques typically depend on an expensive fine-tuning phase to restore the performance lost after parameters are removed. While such retraining is feasible for moderate-sized networks, it becomes prohibitively expensive for LLMs whose scale reaches tens or hundreds of billions of parameters. In addition, realizing practical speedups on modern hardware usually demands that the pruned model adhere to structured sparsity patterns. These patterns enforce a regular arrangement of zeros (e.g., block-wise or group-wise sparsity) that hardware can efficiently exploit during computation. Unfortunately, this regularity often restricts the pruning flexibility, leading to larger accuracy degradation when compared to unstructured pruning, which allows arbitrary weights to be removed. As a result, there exists a fundamental tradeoff: unstructured pruning yields higher accuracy but limited acceleration on commodity accelerators, whereas structured pruning supports efficient execution but typically incurs a higher accuracy penalty.

To address this tradeoff between accuracy retention and hardware-efficient sparsity, structured N:M sparsity [40] has emerged as a compelling middle ground. Under this scheme, each contiguous group of M weights is constrained to contain exactly N nonzero elements. This fine-grained yet regular structure strikes a balance: it preserves more representational capacity than coarse block sparsity, while still offering a predictable sparsity pattern that modern accelerators can readily exploit for speedup. Although N:M sparsity does not reach the accuracy levels attainable with fully unstructured pruning, it consistently delivers substantially better accuracy than other hardware-aligned structured pruning methods, making it a practical and increasingly popular choice for efficient LLM inference. Importantly, many contemporary AI accelerators provide native support for fixed N:M sparse execution [24], making this form of sparsity particularly attractive for efficient LLM inference. Consequently, our work centers on leveraging

Received 11 December 2025; revised 4 March 2026 and 30 March 2026; accepted 17 April 2026. Date of publication 5 May 2026; date of current version 11 June 2026. This work was supported in part by Center for Co-Design of Cognitive Systems (CoCoSys), one of the seven centers in Joint University Microelectronics Program (JUMP) 2.0, a Semiconductor Research Corporation (SRC) Program sponsored by Defense Advanced Research Projects Agency (DARPA). An earlier version of this paper was presented at the International Symposium on Low Power Electronics and Design (ISLPED 2025) [DOI: 10.1109/ISLPED65674.2025.11261770]. *(Corresponding author: Akshat Ramachandran.)*

Akshat Ramachandran and Tushar Krishna are with Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: akshat.r@gatech.edu).

Souvik Kundu, Arnab Raha, Shamik Kundu, and Deepak A. Mathaikutty are with Intel Corporation, Santa Clara, CA 95054 USA.

Digital Object Identifier 10.1109/TVLSI.2026.3686720

1063-8210 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

N:M structured sparsity to enable scalable and performant LLM deployment.

To enable post-training pruning and mitigate fine-tuning cost, SparseGPT [6] proposed to employ row-Hessian information to rank the importance of pretrained weights and prune the least important ones. SparseGPT still required high compute owing to its iterative computation of second-order information (Hessian) for each layer. Wanda [35] avoided the computationally expensive second-order details by introducing a pruning metric that considers both the weight magnitudes and their corresponding input activations (*iActs*). Considering the performance benefits of structured N:M sparsity, both these works [6], [35] extended their approach to support a fixed and predetermined N:M pattern for all the layers. However, only recently [40] researchers have determined that due to the presence of a large number of outliers[1] in LLMs, different layers are inherently heterogeneous and exhibit different tolerances to pruning. Based on this, outlier-weighed layer-wise sparsity (OWL) [40] was proposed. OWL differs from prior techniques in its assignment of different sparsity ratios for different layers through allocations of different N values in the N:M pattern with fixed M. However, this is still suboptimal and has limited representational freedom due to its static selection of M value across all the layers. Moreover, existing hardware accelerators incur significant overhead to employ different values of N and/or M patterns in a single architecture, which was beyond the scope of [40] to address.

It is well established that the decode stage of LLM inference is memory-bound [24], with weights loading constituting a significant bottleneck. Traditional Von-Neumann accelerators consume significant energy and time when moving weights from on-chip buffers to processing elements (PEs). This makes existing traditional accelerators with flexible structured sparsity support [10] poor candidates to accelerate N:M sparse LLMs for inference. An alternative computing paradigm, namely, digital compute-in-memory (DCiM), offers an efficient solution to the above challenge by moving the compute within the memory arrays. This mitigates the memory access bottleneck. However, despite the advantages of DCiM over traditional digital accelerators, there has been limited exploration of flexible structured sparsity support for DCiM-based architectures. This can be associated with the rigid crossbar structure of memory arrays and the infeasibility of transferring the flexible structured sparsity techniques employed for digital accelerators to DCiM (Section III-A).

Our Contributions: We present three key contributions toward addressing the above-mentioned algorithmic and hardware limitations. Specifically, we present a pruning algorithm that can assign different N:M sparsity with a higher degree of freedom across layers and decode time-steps, and then present the first DCiM accelerator to support such patterns with minimal overhead, as summarized below.

We begin by examining how to most effectively assign N:M sparsity across both the layers and the decode steps of an LLM. Our analysis shows that fixing either N or M uniformly across the model imposes unnecessary constraints. In contrast, jointly selecting both parameters on a per-layer basis offers substantially greater representational flexibility, which is critical for maintaining accuracy under structured sparsification. Moreover, motivated by recent findings in reasoning-focused models [large reasoning models (LRMs)] [27] and reinforced by our empirical observations, we further identify that applying a static N or M across all decode steps is similarly suboptimal. Such rigidity fails to capture the dynamic sparsity requirements exhibited during different stages of generation, ultimately limiting the achievable compression ratio.

Motivated by these insights, we propose FLOW—a flexible, layer-wise,outlier-density-aware N:M sparsity selection framework. FLOW builds on a key observation: both the strength of outlier activations and their distribution across layers drive significant heterogeneity in pruning sensitivity. These factors jointly determine how much structured sparsity each layer can tolerate without degrading accuracy. FLOW leverages this heterogeneity by assigning layer-specific N:M configurations and simultaneously selecting both N and M based on each layer's outlier profile. Extending this principle to LRMs, we develop FLOW++, which adapts the sparsity configuration not only across layers but also across decode steps by identifying the dynamic importance of thought types within the chain-of-thought (CoT) [27].

Toward deploying the flexible N:M sparse LLMs in DCiM, we first analyze its challenges in directly incorporating techniques employed by digital accelerators such as [10], particularly the infeasibility of incorporating large multiplexers within each memory cell of DCiM due to the rigid crossbar structure of the memory array. We then introduce a novel DCiM-based accelerator with flexible structured sparsity support—FlexCiM. It decomposes an existing DCiM macro into multiple partitions called submacros (partitioned along the row dimension) and introduces two new units—distribution and merging units. These units coordinate the mapping of weights to each submacro and broadcasting of *iActs* to support diverse N:M sparsity with minimal overheads.

We conduct extensive experiments with foundation models based on both transformer and state space model (SSM) primitives. Specifically, experiments across various LLMs, LRMs, and VLMs demonstrate FLOW outperforming SoTA pruning techniques by achieving up to 36% better accuracy at high sparsity. Our accelerator, FlexCiM, on the other hand, can yield up to 1.75× lower inference latency with 1.5× lower energy compared to the SoTA sparse accelerators with a minimal area overhead of ~6%.

II. Background and Related Works

A. LLM Pruning With N:M Sparsity

N:M sparsity is a popular variant of structured sparsity allowing a model to yield computational and memory benefits in hardware while maintaining improved accuracy compared to traditional structured sparsity (e.g., channel and filter pruning). N:M sparsity retains N nonzero elements within a fine-grained block of consecutive M elements. Existing LLM pruning works, including SparseGPT [6] and Wanda [35], when extended to structured N:M sparsity, were limited

[1]Scalars with larger magnitude compared to the rest of the tensor.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

in their exploration to a fixed N:M pattern for all layers. More recently, Yin et al. [40] highlighted the importance of a heterogeneous sparsity budget for different LLM layers. Specifically, it proposed a nonuniform layer-wise N:M sparsity scheme with different N values for different layers determined based on the number of outliers in each layer. However, unlike existing methods [6], [35], [39], [40], [43], in this work, to yield more optimally pruned models, we present a higher degree of sparse representational freedom by identifying both the optimal N and M values of an N:M pattern.

FLOW [25] introduced a flexible pruning framework that searches over diverse N:M configurations to identify per-layer [30] sparse patterns that best preserve model quality while maximizing hardware efficiency. Building on this idea, we extend FLOW to the emerging domain of LRMs and further push beyond FLOW's spatial-only sparsity formulation by incorporating temporal sparsity, enabling dynamic pruning over decoding steps.

B. Pruning and Compression for LRMs

Recent reasoning-oriented LLMs (e.g., DeepSeek-R1–style models) generate long, explicit chains of thought. Standard weight-pruning methods that match hidden activations on the input prompt often underperform in this regime: they can induce larger accuracy drops and even increase decode-time cost by causing the model to emit more "thinking" tokens [17]. To address this, Lucas et al. [17] proposed reasoning-aware compression (RAC), which augments SparseGPT-style pruning with reconstruction targets drawn from on-policy CoT traces, explicitly aligning pruning with the decode-dominated reasoning workload and recovering much of the lost performance.

In parallel, CoT compression methods act directly on the generated reasoning tokens. Step-entropy-based approaches prune low-information reasoning steps while retaining high-entropy, decision-critical segments of the CoT [14]; Thinking Clearly [3] shows that carefully removing redundant intermediate tokens can improve both efficiency and reasoning robustness over existing KV-compression baselines. ThinKV goes a step further by explicitly decomposing the CoT into distinct thought types using attention sparsity as a signal, and then coupling this semantic structure to a hybrid quantization–eviction policy based on thought importance [27]. This thought-type–aware perspective opens a new direction for reasoning-model pruning: rather than treating all decode steps uniformly, compression policies can be conditioned on the evolving reasoning stage.

C. DCiM Accelerators

Conventional digital accelerators [Fig. 1(a)] [22], [24], [28] employ compute engines separate from memory (buffers) to handle LLM inference, often leading to memory access bottlenecks, increasing the memory access cost [34]. Furthermore, the decode stage of LLM inference is primarily memory-bound, making weight/activation loading a significant bottleneck. In light of these, CiM architectures [7], [34] have been propelled into the mainstream. In CiM, the PEs and weight local memory (SRAM, ReRAM, DRAM, etc.) are merged into a single macro [37], effectively reducing on-chip memory access, yielding accelerated computation. Early work on CiM focused on analog CiM architectures [15], embedding analog multiply-accumulate (MAC) units within memory arrays. While analog CiM architectures demonstrate high energy efficiency, they have various limitations, including vulnerability to noise/process variations, limited bit precision, and significant ADC/DAC overheads, collectively restricting their scalable adoption. Recently, DCiM architectures [Fig. 1(b)] have gained considerable traction, with various industry accelerators [7], [34], [37] adopting these macros to embed digital MAC within memory directly. This eliminates the need for costly DACs/ADCs and offers enhanced robustness against variation and noise, and sufficient precision to achieve high efficiency and accurate inference. In addition, DCiM architectures are inherently compatible with existing digital design flows, making them suitable candidates for LLM acceleration.

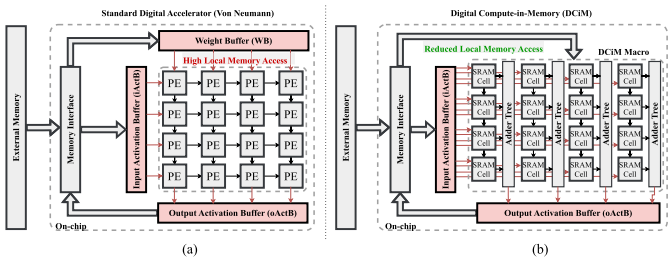


Fig. 1. (a) SA-based Von Neumann digital accelerator [10], [28]. (b) DCiM accelerator. Note, the disjoint weight buffer and compute array in (a) is integrated in (b), reducing local memory access cost.

D. Accelerators Supporting Sparsity

Among digital accelerators, the NVIDIA sparse tensor core [20] exploited fixed 2:4 sparsity of weights in DNNs. VEGETA [10] presented flexible N:M sparsity support for weights, and S2TA [16] extended support for the same for both weights and activations, albeit with significant overheads. Wu et al. [38] proposed hierarchically structured sparsity to represent diverse sparsity patterns in DNNs. Flexible architectures like [23] can potentially accelerate any N:M ratio exploiting unstructured sparsity, yet often suffer from unbalanced critical path delay. However, support for flexible structured sparsity in DCiM accelerators has been largely unexplored. The majority of sparse DCiM accelerators have either focused on bit-level sparsity [42] or unstructured sparsity [33]. Apart from these, a few works [37], [41] introduced fixed 1:2 sparsity support in DCiMs. However, challenges and potential for deploying flexible N:M sparsity in DCiM are yet to be unveiled.

III. Motivation

A. Distribution of Outliers in LLMs

OWL [40] presented a nonuniform layer-wise N:M sparsity with a fixed M and a different N for different layers, with N being proportional to the layer outlier ratio. That is, layers with a high outlier ratio would have a high N value. However,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

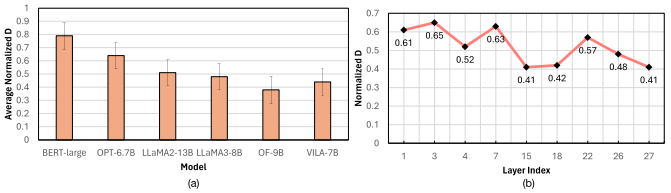


Fig. 2. (a) Outlier distribution measured by pairwise L_1 distance between outliers (normalized to BERT-large) across different models. (b) Intramodel variations in outlier distribution across different layers of a LLaMA3-8B model.

TABLE I

WIKITEXT2 PPL FOR STRUCTURED PRUNING METHODS ACROSS SEVERAL LLM MODEL FAMILIES UNDER VARYING TARGET SPARSITY RATIOS. FLOW-SUPPORTED PATTERNS: (1:2, 1:4, 2:4, 1:8, 2:8, 4:8, AND DENSE) AND OWL-SUPPORTED PATTERNS: (1:8, 2:8, AND 4:8)

Method	N:M	LLaMA-2 [36]			LLaMA-3 [19]		Mixtral [11]	Mamba [9]	
		7B	13B	70B	8B	70B	8x7B	130M	1.4B
Baseline	Dense	5.47	4.83	3.31	6.13	2.85	3.84	20.04	10.42
Target Sparsity: ∼ 50%									
Magnitude	4:8	16.87	9.59	6.83	9.16	5.84	7.91	58.63	43.24
SparseGPT [6]	4:8	8.59	7.18	5.02	8.51	4.34	7.15	29.67	14.98
Wanda [35]	4:8	8.01	6.55	4.68	8.07	4.15	6.98	40.36	21.36
OWL [40]	Mixed N:8	7.92	6.08	4.55	8.02	3.95	6.57	27.28	14.07
FLOW (Ours)	Mixed N:M	**7.04**	**5.73**	**3.97**	**7.03**	**3.46**	**5.96**	**24.48**	**12.96**
Target Sparsity: ∼ 60%									
Magnitude	3:8	54.59	48.96	49.51	52.38	57.63	56.76	1e20	1e14
SparseGPT [6]	3:8	44.68	42.69	39.57	45.09	35.27	41.37	54.83	31.57
Wanda [35]	3:8	43.26	40.24	38.97	44.93	33.78	39.86	1268	1107
OWL [40]	Mixed N:8	20.68	18.68	18.57	16.26	11.23	14.58	31.56	18.67
FLOW (Ours)	Mixed N:M	**18.96**	**16.53**	**17.95**	**15.24**	**10.89**	**13.83**	**28.61**	**15.37**
Target Sparsity: Unconstrained									
OWL [40]	Mixed N:8 (51.3%)	7.98	6.25	4.87	8.36	4.09	6.87	27.35	14.03
FLOW (Ours)	Mixed N:M (58.5%)	**7.13**	**5.95**	**4.57**	**8.03**	**3.87**	**6.35**	**25.12**	**13.08**

varying N with a fixed M value results in suboptimal sparse representation freedom in each layer. We argue that a predetermined M value when assigning different N:M sparsity patterns ignores another key metric in LLM weights, namely the outlier distribution (D). For a tensor l, we compute D^l by a proxy metric that measures the summation of the pairwise distance between the outliers, averaged over the number of outliers. In (1), $dist(.)$ measures the pair-wise L_1-distance between two outliers, where ${}^nC_2$ is the total number of outlier pairs in a layer with n outliers. We then compute the normalized outlier distribution (ND) through the min-max normalization across all the layers as shown in the following equation:

$$ND^l = \frac{\left(D^l - min\left(\mathcal{D}\right)\right)}{max\left(\mathcal{D}\right)}, \text{where} \quad D^l = \frac{\sum_1^{nC_2} dist\left(o_k, o_j\right)}{{}^nC_2} \tag{1}$$

where $\mathcal{D}$ represents the set of Ds for all the layers. To understand the variance of ND over different models, in Fig. 2(a) we plot the average ND averaged over all the layers for different models (including VLMs). Specifically, we observe that models like BERT, OPT have a relatively high average ND, indicating that outliers are sparsely distributed within these models. On the contrary, models like LLaMA3-8B and OpenFlamingo-9B possess a significantly low value of average ND, implying that outliers are largely clustered together. Furthermore, even within the same model, different layers depict varied normalized outlier distributions. Fig. 2(b) demonstrates the diverse ND values for different layers of a LLaMA3-8B. We hypothesize that while choosing the N:M sparsity patterns for different layers, it is important to choose the optimal M value for each layer. If a layer has largely clustered outliers (for example: layer #15 of LLaMA3-8B), the pruning algorithm should favor a larger M value when assigning an N:M sparsity pattern. This will help to: 1) minimize the possibility of pruning outliers within a block of M and 2) have a higher degree of flexibility in deciding which features to prune. We demonstrate the usefulness of such N:M selection in yielding a better perplexity (PPL) score in Table I compared to fixed M.

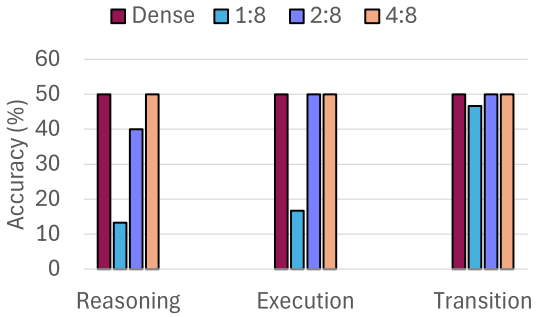


Fig. 3. Accuracy of the model when pruning weights associated with each thought type (R/E/T) using different sparsity patterns.

B. LRM Dynamic Thought Importance

Prior work [27] demonstrates that the CoT produced by LRMs can be systematically decomposed into three distinct thought types of varying importance—reasoning (R), execution (E), and transition (T)—based on their characteristic degrees of attention sparsity. Specifically, Ramachandran et al. [27] show that R thoughts carry the highest importance, followed by E thoughts, with T thoughts contributing the least. Inspired by this hierarchy, we show that the differing importance of R, E, and T thoughts naturally motivates a temporally adaptive assignment of weight sparsity across decode steps, forming the foundation of FLOW++.

For prompts sampled from the AIME dataset [18] using the DeepSeek-R1-Distill-Llama-8B model, we follow the procedure of [27] and compute attention sparsity every 128 decode steps to identify the thought type. To study how thought importance interacts with weight pruning, we consider three fine-grained N:8 sparsity patterns—1:8, 2:8, and 4:8—and evaluate benchmark accuracy when the weights associated with each thought type (R, E, and T) are pruned[2] according to one of these patterns. As shown in Fig. 3, we find a strong alignment between thought importance and pruning tolerance: R thoughts demand larger N (less sparsity) to preserve accuracy, whereas T thoughts remain stable even under the smallest N values, enabling the highest compression for a given M.

C. Challenges of Flexible Structured Sparsity in DCiMs

Sparse digital accelerators [10], [20] have significant multiplexer overhead within each PE to support flexible N:M sparsity. However, this strategy is not directly applicable to DCiM-based accelerators due to various reasons:

[2]For the purposes of this experiment, we only perform masking.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

Area Overhead: The key benefit of DCiM architectures is their high memory density [34], [37]. Typical memory cells in DCiM employ 6T or 12T SRAM cells [7]. However, a single 8:1 multiplexer for flexible N:8 support requires over 42 transistors [37], increasing the area overhead by more than 3× and undermining the compactness and efficiency of the DCiM architectures.

Rigid Crossbar Structure: Digital accelerators using weight-stationary dataflow stream up to M activations into each PE [10], enabling PE-level selection. In contrast, DCiM architectures are restricted by the two-bitline (BL and $\bar{\text{BL}}$) structure of SRAM cells, allowing only two activations to be streamed into a DCiM macro.

Bit-Serial Processing: DCiM architectures employ bit-serial operations to maximize energy efficiency, where each SRAM bit-cell stores a single weight bit and computes partial products with serially streamed activation bits. Von-Neumann accelerators, by contrast, use parallel multipliers/adders and store metadata alongside weights in each PE to manage activation selection. For N:8 sparsity, each N nonzero element requires 3 bits of metadata. In DCiMs, this would necessitate more metadata bits per SRAM cell than weight bits, complicating implementation. These challenges highlight the need for a novel approach to enable flexible N:M sparsity tailored for DCiMs.

IV. FLOW AND FLOW++

In this section, we first formalize our method for outlier identification and then describe FLOW's procedure for optimally selecting layer-wise N:M sparsity patterns. We subsequently extend this framework to the temporal domain with FLOW++, which allocates sparsity adaptively across decode steps.

A. Outlier Identification

In this work, we target N:M sparsity for weights [40] and focus our discussion on weight outliers. Let us assume an LLM weight tensor $W \in \mathbb{R}^{(C_{in} \times C_{out})}$ with *iActs* $X \in \mathbb{R}^{(T*K \times C_{in})}$. Here, T and K are the number of tokens and sequence length, respectively. Inspired by prior works [35], we assign an importance score to each weight element W_{ij}, determined as $\mathcal{I}_W = |W_{ij}| \cdot \|X_j\|_2$ with $\|X_j\|_2$ being the L_2 norm of the *iActs* connected to the weight element. $\mathcal{I}_W$ acts as a measure to identify the outliers based on both weight and associated activation values [35]. Upon obtaining this score, we calculate the mean (μ) and standard deviation (σ) of the weight importance scores for each layer [40]. Subsequently, we categorize all weights exceeding the threshold $\tau \cdot \sigma$ as outliers and the rest as inliers. We empirically identify $\tau = 3$ or $\tau = 5$ as providing the best results.

B. FLOW Methodology

For an L-layer LLM, with a target sparsity ratio of $\mathcal{S}$, the layer-wise sparsity ratios are assigned and handled by two lists $\mathcal{N} = [N^1, N^2, \ldots, N^L]$ and $\mathcal{M} = [M^1, M^2, \ldots, M^L]$. For a layer l, the sparsity pattern is N^l:M^l, the sparsity ratio is

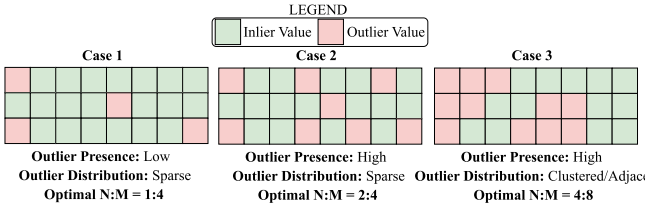


Fig. 4. Example of efficient N:M assignment based on outlier presence and distribution in different situations.

$S^l = (M^l - N^l)/(M^l)$ and $\mathcal{S} = \left(\sum_{l=1}^{l=L} S^l\right)/L$. Following the process outlined in Section IV-A, we obtain the outlier fraction in each layer to get $\mathcal{O} = [O^1, O^2, \ldots, O^L]$, where O^l identifies the normalized outlier fraction in a layer l [40]. Intuitively, if a layer has a larger O^l, the number of nonzeros in a block of M^l elements should be higher (i.e., $N^l \uparrow$). In specific, $N^l \propto O^l$. In Fig. 4, examples with a higher presence of outliers have larger values of N.

FLOW takes the normalized outlier distribution of each layer (ND^l) into account to identify the optimal M^l. Specifically, for each layer weight tensor, we partition it into B nonoverlapping blocks of dimension (128 × 128). Within a block b, we measure D_b^l by calculating the pairwise L_1 distance between all the outlier pairs following (1). We obtain D^l as $\sum_{b=1}^{B} D_b^l$, and finally get ND^l via normalizing the D^l values following (1). A higher ND^l indicates a farther distance between outliers within a layer, implying that they are sparsely distributed (Case 1 in Fig. 4), whereas smaller ND^l indicates a large number of closely located outliers (Case 3 in Fig. 4). Intuitively, if a layer has many closely located outliers, a higher value of M^l is preferred, providing more freedom while performing weight pruning and preventing inadvertent pruning of outliers. In specific, $M^l \propto (1 - ND^l)$.

Automated Layer-Wise N, M Allocation: Given the relations of N^l, M^l, we formulate the problem of assigning layer-wise N:M values as an integer linear programming (ILP) problem

$$\begin{aligned} &\text{argmin} \sum_{l=1}^{L} \left(\alpha \cdot |N^l - k \cdot O^l| + \beta \cdot |M^l - h \cdot \left(1 - ND^l\right)|\right) \\ &\text{s.t.} \ \frac{1}{L}\sum_{l=1}^{L} S^l = S, 0 \le S^l \le 1, \forall l \in \{1, \ldots, L\} \\ &\qquad 1 \le N^l \le M^l, \ N^l \in \{1, 2, 4, 8\}, \quad M^l \in \{2, 4, 8\} \end{aligned} \tag{2}$$

where α and β are the weighing factors to optimize N and M values, respectively. k and h are the hyper-parameters to keep the terms O^l and ND^l in the same ballpark range as N^l and M^l choices, respectively. We empirically choose their values to be $(\alpha, \beta, k, h) = (1, 4, 8, 8)$. This formulation aims to assign values to N^l and M^l for each layer while minimizing deviations from desired characteristics, all while maintaining the target sparsity [see (2)]. N^l is aligned with the normalized outlier count O^l, and M^l is aligned with the outlier distribution ($1-ND^l$). It is important to note that FLOW can be employed to explore any combination of N and M. However, to ensure hardware efficiency, we restrict N and M choices to powers of two and a maximum value of 8 for

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

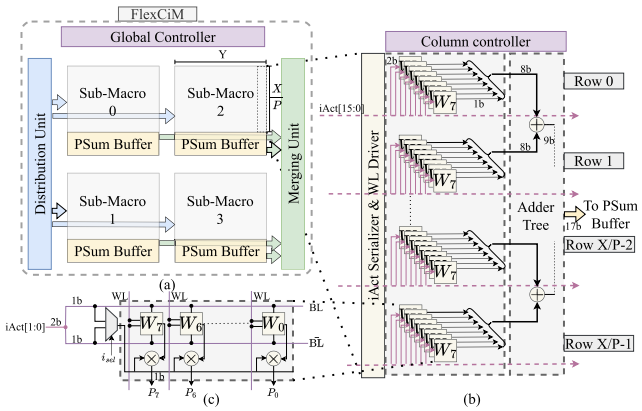


Fig. 5. (a) FlexCiM overview with a partition size $P = 4$. (b) Organization of a single column of a FlexCiM submacro. (c) Memory cell structure.

this work [see N, M choice set mentioned in (2)]. Supporting N, M greater than 8 for LLMs does not yield significant accuracy improvements. Moreover, increasing M enlarges the metadata width and hardware complexity, thereby increasing area and control overhead while offering diminishing returns in compression and accuracy preservation [12].

C. FLOW++ Methodology

After FLOW assigns a per-layer sparsity ratio ($N^l : M^l$), we construct a hierarchical subsparsity set $\mathcal{S}_R^l$ for each layer l corresponding to the R thought type. For example, if FLOW selects a 4:8 sparsity pattern for a given layer, we derive submasks such as 1:8, 2:8, and 4:8, where each ratio preserves a subset of the nonzero positions defined by the original 4:8 configuration. Using calibration samples (Section VI), we apply the thought-type decomposition of [27] to identify R, E, and T segments across decode steps. For the R thought type, we then select the sparsity ratio from $\mathcal{S}_R^l$ that minimizes the output reconstruction error [32].

Once the optimal R-level sparsity ratio is chosen, we construct an analogous hierarchical subsparsity set $\mathcal{S}_E^l$—restricted to ratios no denser than the selected R pattern—and choose from it the best sparsity level for E thoughts. Finally, we generate the subsparsity set $\mathcal{S}_T^l$, constrained to be no denser than the chosen E pattern, and select the optimal sparsity ratio for T thoughts. This sequential procedure enforces the desired hierarchy $(N_R^l/M_R^l) \geq (N_E^l/M_E^l) \geq (N_T^l/M_T^l)$, in line with our observations in Section III-B, while preserving consistency with the base ratio determined by FLOW. As we detail later, this construction enables seamless switching between sparsity levels during generation.

V. FlexCiM Accelerator

Overview: Fig. 5(a) provides an overview of the FlexCiM architecture. FlexCiM extends an existing $X \times Y \times 8$ [37] SRAM-based DCiM bit-cell macro supporting fixed 1:2 sparsity, where (X, Y) represent the crossbar array dimensions with the memory word size being 8-bits. FlexCiM can be employed to accelerate any layer that can be represented as a GEMM [10] operation. It partitions the existing macro into P components (named as submacros), each of dimension $X/P \times Y \times 8$. Following [34], [37], we employ a two-level memory hierarchy: a level-1 SRAM that feeds data directly to the accelerator and a level-2 global shared SRAM that loads data from an off-chip DRAM. To seamlessly support diverse N:M sparsity patterns with this architecture, we introduce two novel hardware components, namely, the distribution and the merging units, to orchestrate the working of the P submacros. In our demonstrated implementation, we assume $X = 128$, $Y = 32$ for the DCiM bit-cell macro, with partition size of $P = 4$, resulting in an 8-kb submacro of dimensions $32 \times 32 \times 8$. Each submacro column is divided into 32 banks to enable parallel MAC operation. To enable the bit-serial computation, a local iAct buffer feeds the activations to the distribution unit (detailed later) at a rate of X 8-bit activations per cycle, which is serialized via the iAct serializer unit in each submacro [7], [34]. We employ a two-tier control unit comprising a global controller and column-wise controller to manage sparsity metadata, generate control signals that manage pipelining across rows and columns of submacros, and configure multiplexer select signals. The weights corresponding to the per-layer sparsity pattern, along with their metadata, reside in the shared global SRAM. During decode, as the thought type changes based on the attention sparsity level (computed on the host for simplicity), the global controller receives the appropriate signal and computes the metadata for the corresponding subsparsity pattern and loads it into the DCiM macro. Since all thought-specific sparsity patterns are hierarchical subsparsities of the global pattern, this selection is straightforward and requires only indexing into the precomputed subset of nonzeros, without any additional recomputation or data movement.

A. Submacro Architecture

Memory Cell: Fig. 5(c), presents a memory word in FlexCiM, with each word composed of 8 bit-cells. To abstract the circuit complexity and demonstrate our proposal to be orthogonal to any SRAM technology, we implement a 28 nm, standard cell, latch-based memory structurally resembling a 6T SRAM cell [37]. Each memory cell has two bit-lines BL, $\overline{\text{BL}}$, shared by all cells in a row, and a word-line (WL), shared per column. All memory cells in a column share a control signal to enable the compute units, namely, *EN_COL*. Inspired by [37], we implement a 2:1 multiplexer that is shared by a memory word to select one of the two *iActs* (via control signal (i_{sel}) streamed along the two bit-lines to enable 1:2 structured sparsity as a baseline case. A bit-serial multiplier within each SRAM cell (implemented as NOR gate) performs the multiplication between the selected iAct and stored weight. The memory cell has two modes of operation: memory ($\mathcal{M}_S$) and compute ($\mathcal{C}$) mode. Standard read and write operations are performed in $\mathcal{M}_S$ mode. The WL is activated to access all cells in a column, with BLs used to read/write data. $\mathcal{C}$ mode is enabled for all cells in a column only when WL = 0 and the corresponding *EN_COL = 1*, allowing the compute units to perform MAC operations.

Memory Column: Fig. 5(b) illustrates the organization of a column in a submacro. Each column comprises 32 rows of

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

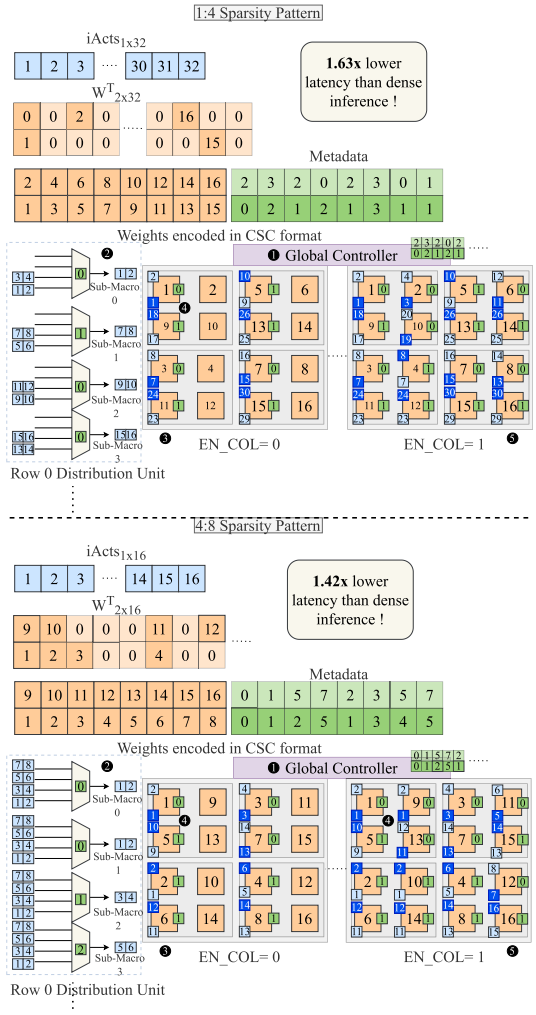


Fig. 6. Example of FlexCiM running 1:4 and 4:8 sparsity patterns.

8-bit memory words and a column-wise adder tree. Each memory word performs bit-wise multiplication with the streamed iAct bits (MSB first), which are then accumulated column-wise by a 32-input adder tree and directed to the partial sum (PSum) accumulator.

PSum Accumulator: Since DCiMs typically perform bit-serial arithmetic, the MACs from each column-wise adder tree must be bit shifted based on the bit significance of the streamed iAct and subsequently accumulated to compute the final PSum. The complete operation can be expressed as, $\text{PSum}_{\text{final}} = \sum_{i=0}^{i=7} 2^i \times \sum(iAct[i] \cdot W)$.

Column-Wise Controller: Each submacro possesses a controller that is dedicated to generating the i_{sel} signal for all the 2:1 multiplexers in each column and enabling column pipelining (see Section V-C) via the *EN_COL* signal.

B. Hardware Extensions for Flexible N:M Sparsity

N:M Sparsity Storage Format: In FlexCiM, all sparse weight tensors are encoded in CSC format [10]. Each column is stored as a list consisting of all the nonzero values, and the coordinates of the N nonzero values within a block of size M are stored as the corresponding metadata (see Fig. 6).

We introduce two key hardware blocks, the distribution unit and merging unit, that enable flexible N:M structured sparsity together.

Distribution Unit: The distribution unit is responsible for efficiently feeding *iActs* to the submacros based on the N and M values. As depicted in Fig. 5, it is composed of $PP : 1$ multiplexers that serve each of the P submacros ($P = 4$ in our implementation) with a bit-width of 16 bits (packing two 8-bit *iActs*) per input line. A distribution unit is shared by spatially identical rows of all submacros. For example, row 0 of all P submacros shares one distribution unit. The M value indicates the number of *iActs* to be distributed by each multiplexer in the distribution unit. The N value of a sparsity pattern identifies the number of submacros that are aggregated together to process the same block M. In particular, for $N > 1$, multiple multiplexers will have the same set of inputs. Still, the distribution unit will select the appropriate input line based on the metadata (see Section V-D for a demonstration). The distribution unit abstracts the complexity of supporting large multiplexers in the memory cell by efficiently selecting the correct set of two activations to be fed to the memory array. We demonstrate in Section V-C that by efficiently performing row and column pipelining, we require only 32 distribution units serving all rows of a column of all submacros every clock cycle.

Merging Unit: The merging unit is responsible for accumulating the PSums that are individually computed by each column of each submacro to generate the final PSum in our partitioned scheme. To enable this, the merging unit comprises a P-input adder tree that reads its inputs from the PSum accumulator's buffer.

C. Pipelining Scheme

Following on-chip buffers employed in comparable real-world systems [5], [34], we design the local iAct buffer with a bandwidth of 1024 bits/cycle (128 8-bit activations). For dense LLM inference, 128 *iActs* are streamed to a column, enabling parallel MAC operations by all rows in a column of all submacros. However, for sparse inference with the highest sparsity, i.e., 1:8 in our implementation, each row requires eight *iActs* to choose from, which translates to 1024 *iActs* required by each column. Considering the upper limit of memory bandwidth, during 1:8 inference, only four rows of a column can perform parallel MAC operations. Similarly, eight rows for 1:4 and 16 rows for 1:2. Due to the bit-serial nature of DCiMs, a MAC operation takes eight cycles. Therefore, we overlap computation with memory access via row-pipelining. The number of row pipeline stages ($\#_{stages}$) is equal to 32/(# of rows grouped). For example, for the 1:8 case, each set of four rows in a column—activated via *EN_COL*—are fed with *iActs* in a single cycle, and the remaining 28 rows are fed *iActs* in the subsequent eight clock cycles in sets of 4, resulting in $\#_{stages} = 8$. Every $\#_{stages}$ cycle, the *EN_COL* of the adjacent column is activated, and similarly, *iActs* are fed to all rows in the column. Once all columns have been fed with *iActs*, *EN_COL* of the first column is reactivated, and the process is repeated. We perform column-pipelining to reduce the overhead of the distribution unit because, without

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

column pipelining, each column would require a dedicated distribution unit, which would be prohibitively expensive for memory-centric designs.

D. Walkthrough Example

To demonstrate FlexCiM enabling inference with diverse N:M patterns, we explain through two simple examples in Fig. 6. The N of an N:M pattern identifies the number of submacros that work together to process a block of size M. Similarly, M specifies the number of *iActs* handled by each multiplexer in the distribution unit. For the 4:8 pattern, each set of four nonzero values is mapped to spatially identical locations in four submacros. For the 1:4 pattern, each submacro works independently on a block of size M, with each multiplexer receiving a separate set of four *iActs*. Note that the same distribution units employed for *iActs* can be used to map the weights to each of the submacros.

① The global controller receives the CSC metadata (in green) of the mapped weights and generates corresponding select signals to control the multiplexers. The LSB of the metadata is always reserved for the 2:1 multiplexer in the submacro. The remaining bits are employed for the multiplexers in the distribution unit. For example, for M = 8, the metadata requires 3 bits, where the top 2 bits are for the distribution unit multiplexers.

② The distribution unit for each row (only row 0 unit shown in the figure) in column 0 selects the appropriate input line and directs it to the corresponding submacros.

③ The first column of each of the submacros receives the selected *iActs*. These are then serialized and sent along the bit-lines.

④ The 2:1 multiplexer selects the appropriate iAct (highlighted in dark blue) from the bit-lines to be multiplied with the stored weights.

⑤ After $\#_{stages}$ cycles, column 1 receives the *iActs* and the process is repeated. The merging unit performs final PSum accumulation by reading from the PSum buffers of each column as they are calculated. Notably, in the 1:2 base case and dense inference, no selection occurs in the distribution units. Furthermore, for dense inference, the same *iActs* are streamed along both bit-lines, and I_{sel} is a don't-care.

VI. Results and Discussions

A. Experimental Setup

Models and Datasets: We evaluate transformer-based LLMs and VLMs [2], [11], [19], [36], and also include Mamba-based SSMs [26] for algorithm benchmarking. For evaluating FLOW++, we employ two LRMs, DeepSeek-R1-Distill-Llama-8B and NVIDIA AceReason-14B. For LLM calibration, we use 256 samples from PILE [8], and for LRM calibration, we employ 256 samples from the s1K dataset [21]. Models are compared by PPL on WikiText2 and accuracy on several downstream tasks.

Quantization Implementation: Following prior work [34], [37], we quantize weights and activations to 8-bit integer (INT8) representations with a group size of 128. Scaling is performed using a group-wise E8M0 format [24]

Algorithm Implementation: We implement FLOW and FLOW++ in PyTorch. All experiments are conducted using a single NVIDIA H100 GPU. The runtime for the complete process of FLOW and FLOW++ is less than 65 min. The overall algorithm complexity scales with both the dimensionality of each layer and the number of layers, resulting in approximately linear growth with model depth and parameter size.

Algorithm Baselines: We employ standard magnitude-based pruning as the naive baseline, and three SoTA pruning techniques, namely, SparseGPT [6], Wanda [35], and OWL [40], to compare with FLOW.

Accelerator Implementation: The FlexCiM accelerator is implemented in Verilog RTL and synthesized, placed, and routed using Synopsys Design Compiler and Cadence Innovus, respectively, using TSMC 28-nm technology library. Recent silicon prototypes demonstrate that fully synthesizable DCiM designs implemented using standard-cell flows can accurately capture first-order PPA characteristics without relying on custom memory hard macros [1]. In particular, Agarwal et al. [1] show that latch-based, synthesizable DCiM macros integrated through conventional synthesis and place-and-route flows provide reliable area and energy estimates. Our RTL-based implementation follows this methodology to ensure fair comparison and flexibility. While deeper exploration of DCiM unit sizing and utilization tradeoffs is valuable future work, we do not expect such refinements to materially alter the relative conclusions of FlexCiM, as the added flexibility primarily impacts peripheral control and routing logic rather than the core compute-in-memory datapath. FlexCiM achieves a peak clock frequency of 1 GHz. We assume all model parameters fit within the global SRAM for simplicity [34]. We use CACTI to estimate the area and power of level-1 and level-2 SRAMs. For end-to-end performance and energy metrics, we design a cycle-accurate simulator based on DNNWeaver [31], following prior works [28].

Accelerator Baselines: We compare FlexCiM against SoTA sparse digital accelerator VEGETA [10], 1:2 structured sparse DCiM-based accelerator (SDP), dense systolic array (SA), and dense DCiM. To ensure a fair comparison, we implement configurations of all accelerators so that they achieve iso-throughput. All digital accelerators are implemented with 64×64 array configuration to achieve the same peak throughput as DCiM-based designs with $128 \times 32 \times 8$ configuration. All designs are scaled to 28 nm using DeepScale [29].

B. FLOW: Results and Analysis

PPL Benchmark: In Table I, we compare FLOW with the baselines for different target sparsity ratios. Magnitude, SparseGPT, and Wanda require a static determination of the N:M pattern. We therefore assign 4:8 and 3:8 as the sparsity pattern for 50% and 60% target sparsity, respectively. Since OWL can explore different values of N for a fixed M, we fix M = 8. Assigning M smaller than 8 for these methods results in poor performance. FLOW significantly outperforms the baselines, at all target sparsity ratios, achieving up to 18% PPL improvement (PPL ↓) over OWL, SparseGPT, and Wanda. The benefit of FLOW is evident at higher target sparsity

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

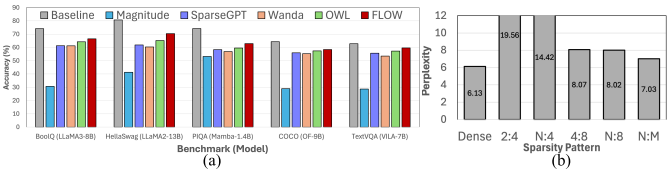


Fig. 7. (a) Accuracy (%) comparison across four zero-shot tasks with 60% target N:M sparsity across different LLMs and VLMs. (b) Ablation study on the impact of different N:M patterns.

TABLE II

COMPARISON OF LRM ACCURACY ON REASONING BENCHMARKS. FLOW– AND FLOW++ SUPPORTED PATTERNS: (1:2, 1:4, 2:4, 1:8, 2:8, 4:8, AND DENSE)

AIME (Global Sparsity Budget 50%)			
Method	**Avg. Sparsity (%)**	**R1-Llama-8B**	**AceReason-14B**
Baseline	–	50.0	73.33
OWL	50.0	43.3	63.33
FLOW	50.0	43.3	66.67
FLOW++	60.25	43.3	66.67
LiveCodeBench (Global Sparsity Budget 50%)			
Method	**Avg. Sparsity (%)**	**R1-Llama-8B**	**AceReason-14B**
Baseline	–	32.14	57.8
OWL	50.0	28.95	53.6
FLOW	50.0	29.8	55.6
FLOW++	63.5	28.7	55.1

ratios, demonstrating that the higher sparse representational freedom offered by FLOW is instrumental in achieving better performance.

Furthermore, we compare the performance of an FLOW-generated model with that generated via OWL for unconstrained target sparsity. Notably, not only does FLOW result in a higher compression (1.14×), but it also achieves a lower PPL than OWL across all models.

Zero-Shot Performance Benchmark: We evaluated the zero-shot performance of the pruned models in Fig. 7(a) at a target sparsity ratio of 60%. Specifically, we performed this evaluation with both LLMs and VLMs to study the generalization of FLOW across modalities. Notably, FLOW outperforms all other baselines across all tasks with an accuracy improvement of up to 36%.

LRM Benchmark Performance: We evaluated LRM performance on AIME and LiveCodeBench to assess FLOW++ across diverse reasoning tasks and model families. As shown in Table II, with a global sparsity budget of 50%, FLOW consistently outperforms OWL. Moreover, introducing temporal sparsity through FLOW++ enables up to a 13.5% higher effective sparsity ratio (by performing higher compression across various decode stages) while retaining near-lossless accuracy relative to FLOW.

FLOW Ablation: We evaluate the impact of different N:M patterns on the PPL with an LLaMA3-8B model in Fig. 7(b) for target sparsity of 50%. To demonstrate the impact of static N:M choices, we performed ablations with two N:M patterns, namely 2:4 and 4:8, where the weights are pruned based on their importance scores. We find that the model pruned with 2:4 sparsity yields a significant increase in PPL over the dense baseline. 4:8, though it improves the performance, still falls significantly behind the baseline dense. However, enabling flexibility for N, i.e., N:4 = {1:4, 2:4, and 4:4 (dense)}, alleviates some of the PPL degradation from a static N:M. More importantly, enabling flexibility for both N and M as proposed in FLOW, i.e., N:M = {1:2, 1:4, 2:4, 1:8, 2:8, 4:8, 8:8 (dense)}, we achieve the lowest PPL that is closest to the dense baseline.

This trend stems from the interaction between block granularity and the spatial distribution of outliers within each layer. Consider a layer where outliers are widely distributed rather than tightly clustered. In a static N:8 scheme, where N is selected (e.g., N=2) to satisfy a target sparsity, only two weights can be retained per 8-element block regardless of how outliers are positioned locally. If multiple outliers fall within the same block, some must be pruned despite meeting the global sparsity constraint. In such cases, a smaller block size (e.g., 1:4) can better preserve distributed outliers by reducing destructive coupling within large blocks. More generally, a flexible scheme can increase N when outlier density is high or reduce M when outliers are spatially dispersed, thereby aligning sparsity with both outlier presence and distribution. This joint adaptation of N and M mitigates rigid block effects and leads to lower PPL than static N:M or Mixed N:8 schemes.

TABLE III

COMPARISON OF FLEXCIM WITH BASELINES AT 28NM. ✓, ✗ IN A COLUMN IDENTIFY THE PRESENCE AND ABSENCE OF A FEATURE, RESPECTIVELY

Architecture	**Component** (Area (mm^2))	**Total Area** (mm^2)	**Flexible N:M**	**Flexibility Overhead**	**Compute Density** (Peak TOPS/mm^2)
VEGETA [10]	Input/Output buffers (0.27) Weight buffer (0.18) PE array (2.46) Sparsity support (0.38)	3.28	✓	15.4%	2.36
SDP [37]	Input/Output buffers (0.27) Weight buffer (0) DCiM array (0.68) Sparsity support (0.024)	0.98	✗	3.5%	7.65
FlexCiM (Ours)	Input/Output buffers (0.27) Weight buffer (0) DCiM array (0.72) Sparsity support (0.043)	1.03	✓	5.9%	7.28

C. FlexCiM: Results and Analysis

Area Comparison: In Table III, we compare the accelerator area breakdown of VEGETA and SDP with FlexCiM. All designs have the same buffer configurations. SDP and FlexCiM have zero weight buffer area because these designs replace the weight memory and compute array of digital accelerators with DCiM macros. FlexCiM and SDP have significantly lower area than VEGETA due to the high memory density of DCiM-based designs. Specifically, FlexCiM, despite supporting flexible N:M sparsity compared to SDP's fixed 1:2 sparsity, has a modest compute area overhead of 5.9% and similar compute density. Furthermore, FlexCiM's partitioned architecture reduces the long chain of column-wise adder trees by having multiple smaller adder trees across submacros. Importantly, FlexCiM's architecture requires no additional overhead compared to a similar N:8 design, since the same distribution and merging units are inherently required by the partitioned DCiM dataflow. Even if restricted to a static M, the hardware organization remains unchanged, meaning flexibility in N and M comes without commensurate overhead.

Performance Comparison: In Fig. 8, we conduct a performance comparison across different models pruned via FLOW.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

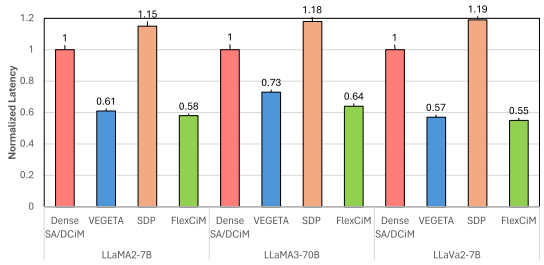


Fig. 8. Normalized performance comparison between different dense and sparse accelerators for different models.

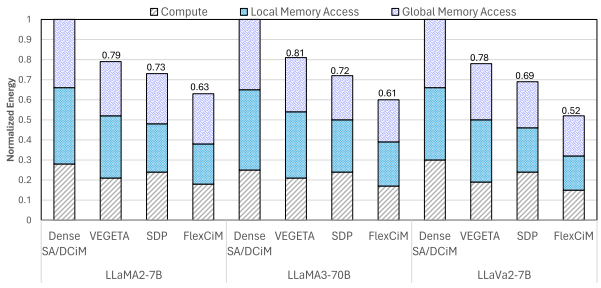


Fig. 9. Normalized energy comparison between several digital and DCiM accelerators and the proposed FlexCiM across different models for flexible N:M inference acceleration.

We report normalized latency numbers, normalized to a dense SA/DCiM. Since we assume iso-peak-throughput, the dense SA and DCiM have the same performance. Since SDP supports fixed 1:2 sparsity or dense, it has a higher latency than dense accelerators because FLOW assigns a 1:2 pattern to very few layers, and in SDP, all other N:M patterns are treated as dense. FlexCiM achieves a significantly lower latency up to 1.72× than the baselines.

Energy Comparison: In Fig. 9, we compare the normalized energy comparison of different accelerators running various models pruned via FLOW. We report energy breakdown in terms of local memory and global memory access, where local memory access measures energy from the Level-1 SRAM for DCiM designs and the weight activation buffer for digital accelerators. The global memory access measures the energy consumption of data access from the shared SRAM. Since off-chip memory access will be approximately similar for DCiM or digital accelerators, we do not explicitly highlight it. Following [34], we assume all model parameters fit in the global SRAM and consider DRAM access as out of scope for this work. FlexCiM has the lowest energy consumption across different models. Particularly, the DCiM-based architecture significantly reduces the energy contribution from local memory accesses. FlexCiM has up to 1.5× lower energy consumption compared to all baselines across different models.

Host-Controller Communication: Transferring the computed attention scores to the host and receiving the corresponding sparsity configuration back incurs on the order of 10–20 μs per token when amortized over 128 decode steps (1.5–2-ms host–device communication on average) [4]. Moreover, this communication can be efficiently overlapped with ongoing decode computation, effectively hiding it with minimal impact on end-to-end latency.

VII. Conclusion

We show that the distribution of outliers across layers drives their nonuniform sensitivity to sparsity, motivating flexible per-layer N:M choices. FLOW addresses this by selecting optimal N and M from a diverse search space using outlier-aware scoring, yielding up to 36% accuracy improvement, 18% PPL reduction, and higher compression than prior methods. FLOW++ extends these gains to LRMs via temporal sparsity, enabling up to 13.5% higher effective sparsity with near-lossless accuracy. To accelerate such heterogeneous patterns, we introduce FlexCiM, a DCiM-based architecture supporting flexible N:M sparsity, achieving up to 1.75× lower latency and 1.5× lower energy over state-of-the-art accelerators.

References

[1] A. Agarwal, S. K. Hsu, M. A. Anders, A. Raha, D. A. Mathaikutty, and R. Krishnamurthy, "A 147TOPS/W, 250TOPS/mm^2, fully synthesizable, digital compute-in-memory accelerator supporting INT8×INT8 with zero-point quantization in Intel 18A technology," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2026, pp. 528–530.

[2] A. Awadalla et al., "OpenFlamingo: An open-source framework for training large autoregressive vision-language models," 2023, *arXiv:2308.01390*.

[3] D. Choi et al., "Thinking clearly: Enhancing reasoning with redundant token pruning," in *Proc. Findings EMNLP*, 2025, pp. 21437–21451.

[4] W. Deng et al., "HGCA: Hybrid GPU-CPU attention for long context LLM inference," 2025, *arXiv:2507.03153*.

[5] F. Farshchi, Q. Huang, and H. Yun, "Integrating NVIDIA deep learning accelerator (NVDLA) with RISC-V SoC on FireSim," in *Proc. 2nd Workshop Energy Efficient Mach. Learn. Cognit. Comput. Embedded Appl. (EMC2)*, Feb. 2019, pp. 21–25.

[6] E. Frantar and D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 10323–10337.

[7] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[8] L. Gao et al., "The pile: An 800GB dataset of diverse text for language modeling," 2021, *arXiv:2101.00027*.

[9] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.

[10] G. Jeong et al., "VEGETA: Vertically-integrated extensions for sparse/dense GEMM tile acceleration on CPUs," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Feb. 2023, pp. 259–272.

[11] A. Q. Jiang et al., "Mixtral of experts," 2024, *arXiv:2401.04088*.

[12] M. Lee, A. Ramachandran, and T. Krishna, "Recap: Training-free compensation for coarse activation channel pruning in compressed LLMs," *Mach. Learn. Comput. Archit. Syst.*, Jan. 2025.

[13] M. Lee, A. Ramachandran, S. Kundu, and T. Krishna, "Polestar-cache: Reconciling parallel decoding and accuracy in diffusion LLMs via token drift-aware KV cache recalibration," in *Proc. Workshop Latent Implicit Thinking–Going Beyond CoT Reasoning*, Jan. 2025.

[14] Z. Li et al., "Making slow thinking faster: Compressing LLM chain-of-thought via step entropy," 2025, *arXiv:2508.03346*.

[15] Q. Liu et al., "A fully integrated analog ReRAM based 78.4 TOPS/W compute-in-memory chip with fully parallel MAC computing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 500–502.

[16] Z.-G. Liu, P. N. Whatmough, Y. Zhu, and M. Mattina, "S2TA: Exploiting structured sparsity for energy-efficient mobile CNN acceleration," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Seoul, South Korea, Apr. 2022, pp. 573–586.

[17] R. Lucas, K. Behdin, Z. Wang, Q. Song, S. Tang, and R. Mazumder, "Reasoning models can be accurately pruned via chain-of-thought reconstruction," 2025, *arXiv:2509.12464*.

[18] (2024). *Aime 2024 Problems*. [Online]. Available: https://artofproblemsolving.com/wiki/index.php/2024_AIME_I_Problems

[19] *Introducing Meta LLAMA 3: The Most Capable Openly Available Llm To Date*, Meta AI, Menlo Park, CA, USA, 2024.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.

[20] A. Mishra et al., "Accelerating sparse deep neural networks," 2021, *arXiv:2104.08378*.
[21] N. Muennighoff et al., "S1: Simple test-time scaling," 2025, *arXiv:2501.19393*.
[22] A. Parashar et al., "SCNN: An accelerator for compressed-sparse convolutional neural networks," *ACM SIGARCH Comput. Archit. news*, vol. 45, no. 2, pp. 27–40, 2017.
[23] A. Raha, D. A. Mathaikutty, S. Kundu, and S. K. Ghosh, "FlexNPU: A dataflow-aware flexible deep learning accelerator for energy-efficient edge devices," *Frontiers High Perform. Comput.*, vol. 3, Jun. 2025. [Online]. Available: https://www.frontiersin.org/journals/high-performance-computing/articles/10.3389/fhpcp.2025.1570210
[24] A. Ramachandran, S. Kundu, and T. Krishna, "MicroScopiQ: Accelerating foundational models through outlier-aware microscaling quantization," in *Proc. 52nd Annu. Int. Symp. Comput. Archit.*, Jun. 2025, pp. 1193–1209.
[25] A. Ramachandran, S. Kundu, A. Raha, S. Kundu, D. K. Mathaikutty, and T. Krishna, "Accelerating LLM inference with flexible N: M sparsity via a fully digital compute-in-memory accelerator," in *Proc. ISLPED*, Aug. 2025, pp. 1–7.
[26] A. Ramachandran, M. Lee, H. Xu, S. Kundu, and T. Krishna, "OuroMamba: A data-free quantization framework for vision mamba," 2025, *arXiv:2503.10959*.
[27] A. Ramachandran, M. Neseem, C. Sakr, R. Venkatesan, B. Khailany, and T. Krishna, "ThinKV: Thought-adaptive KV cache compression for efficient reasoning models," 2025, *arXiv:2510.01290*.
[28] A. Ramachandran, Z. Wan, G. Jeong, J. Gustafson, and T. Krishna, "Algorithm-hardware co-design of distribution-aware logarithmic-posit encodings for efficient DNN inference," 2024, *arXiv:2403.05465*.
[29] S. Sarangi and B. Baas, "DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.
[30] J. Seo, A. Ramachandran, Y.-C. Chuang, A. Itagi, and T. Krishna, "AIRCHITECT v2: Learning the hardware accelerator design space through unified representations," in *Proc. Design, Autom. Test Eur. Conf. (DATE)*, Mar. 2025, pp. 1–7.
[31] H. Sharma et al., "Dnnweaver: From high-level deep network models to FPGA acceleration," in *Proc. Workshop Cognit. Architectures*, 2016.
[32] S. Shin, W. Park, J. Lee, and N. Lee, "Rethinking pruning large language models: Benefits and pitfalls of reconstruction error minimization," 2024, *arXiv:2406.15524*.
[33] A. Sridharan, F. Zhang, J.-S. Seo, and D. Fan, "SP-IMC: A sparsity aware in-memory-computing macro in 28nm CMOS with configurable sparse representation for highly sparse DNN workloads," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.
[34] H. E. Sumbul, J.-S. Seo, D. H. Morris, and E. Beigne, "A fully digital and row-pipelined compute-in-memory neural network accelerator with system-on-chip-level benchmarking for augmented/virtual reality applications," *IEEE Micro*, vol. 44, no. 2, pp. 61–70, Mar. 2024.
[35] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," in *Proc. ICLR*, 2024, pp. 4942–4964.
[36] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
[37] F. Tu et al., "SDP: Co-designing algorithm, dataflow, and architecture for in-SRAM sparse NN acceleration," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 1, pp. 109–121, Jan. 2023.
[38] Y. N. Wu, P.-A. Tsai, S. Muralidharan, A. Parashar, V. Sze, and J. Emer, "HighLight: Efficient and flexible DNN acceleration with hierarchical structured sparsity," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2023, pp. 1106–1120.
[39] L. Yin, S. Liu, A. Jaiswal, S. Kundu, and Z. Wang, "A task-centric angle of LLM pre-trained weights through sparsity," in *Proc. ICML*, 2024.
[40] L. Yin et al., "Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMS to high sparsity," in *Proc. ICML*, 2024.
[41] J. Yue et al., "A 2.75-to-75.9 TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 238–240.
[42] Y. Zhan, W.-H. Yu, K.-F. Un, R. P. Martins, and P.-I. Mak, "A 28-nm 18.7 TOPS/mm^2 89.4-to-234.6 TOPS/W 8b single-finger eDRAM compute-in-memory macro with bit-wise sparsity aware and kernel-wise weight update/refresh," *IEEE J. Solid-State Circuits*, vol. 59, no. 11, pp. 3866–3876, Nov. 2024.
[43] Y. Zhang et al., "Dynamic sparse no training: Training-free fine-tuning for sparse LLMs," 2023, *arXiv:2310.08915*.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:13 UTC from IEEE Xplore. Restrictions apply.